

CSC-455: Mobile Application Development - Viva Cheatsheet

This cheatsheet summarizes the key concepts and components covered in the course outline for quick review.

1. Android Fundamentals (Weeks 01 & 05)

Topic	Key Concepts	Notes for Viva
Android Architecture & Runtime	Linux Kernel, HAL, ART/Dalvik, Libraries.	ART (Android Runtime): Executes Dalvik bytecode, responsible for application component execution in a sandboxed environment.
First App	Activity, Manifest, Resources.	Manifest: XML file describing app components, permissions, and features to the OS.
Intents	Explicit vs. Implicit.	Explicit: Launches a known component (e.g., <code>Activity.class</code>). Implicit: Requests the system to find a component that can handle a specific action (e.g., opening a camera, sending an email).
Intent Filter	Action, Data, Category.	Specifies what types of Intents a component can receive, allowing the system to match Implicit Intents to your component.

2. Traditional UI & Layouts (Week 02)

Layout	Purpose	weight / gravity	Hierarchy Performance
Constraint Layout	Complex, flat UI. Position based on constraints (relative to parent/siblings).	N/A	Excellent. Recommended root layout to minimize nesting.
Linear Layout	Arranges components in a single row (<code>horizontal</code>) or column (<code>vertical</code>).	weight : Proportional space distribution along the single axis. gravity : Position content within the view.	Good for simple, linear structures.
Relative Layout	Positions views relative to each other or the parent boundaries.	N/A	Legacy, generally replaced by Constraint Layout.
Widgets	TextView, EditText, Button.	Basic building blocks for text display, user input, and user action triggers.	

3. Jetpack Compose (Weeks 03 & 04)

Topic	Key Concepts	Notes for Viva
-------	--------------	----------------

Introduction to Compose	Composable Function, Declarative UI.	Kotlin functions marked with <code>@Composable</code> that describe the UI's appearance based on state. Replaces XML layouts.
Comparison	XML (Imperative) vs. Compose (Declarative).	Imperative: Manually find <code>View</code> and call setters (<code>findViewById</code> , <code>setText</code>). Declarative: Describe the UI; Compose handles update logic (Recomposition).
UI Components	Text, Button, Image, TextField.	Standard composables replacing traditional Widgets.
Layouts	Column, Row, Box.	Column: Vertical arrangement. Row: Horizontal arrangement. Box: Stacking/overlapping elements. Modifiers control layout behavior.
State Management	<code>remember</code> , <code>mutableStateOf</code> , State Hoisting, ViewModel.	State Hoisting: Moving state from a Composable to its caller to make the Composable stateless, reusable, and testable.
Navigation	Navigation Component, NavHost.	Manages the back stack of Composable destinations within a single Activity.

4. Persistence & Background Work (Weeks 05, 06, 07, 08, 09)

Topic	Key Concepts	Notes for Viva
State Preferences	Shared Preferences.	Simple, small key-value storage (e.g., user settings, flags). Easy to use but not for complex data.
File Storage	Reading/Writing.	Used for unstructured data (e.g., large images, documents). Can read from raw assets or online sources.
SQLite (Built-in DB)	Relational Database.	Used for large, structured, relational data. Requires complex querying and indexing. Room is the recommended abstraction layer.
Coroutines	<code>suspend functions</code> , <code>Dispatchers</code> , <code>Types</code> .	Lightweight solution for asynchronous programming, making non-blocking code look sequential (avoids callback hell). Used heavily for IO tasks (networking, DB).
Services	Foreground Services.	Runs operations in the background. Foreground Service: Must be noticeable to the user (e.g., music player) and requires a persistent notification.
WorkManager	Worker, WorkRequest (OneTime/Periodic), Constraints.	Worker: Contains the actual deferred code. WorkRequest: Defines <i>how</i> and <i>when</i> the work runs. Ensures reliable execution, even if the app closes or the device restarts.
UI Notifications	Notification Channel (Android 8.0+), Builder.	Used to alert the user about events outside the application's UI. Channels are mandatory for categorization and user control.

5. Data Sharing & Networking (Weeks 10 & 11)

Topic	Key Concepts	Notes for Viva
Content Providers	URI, Query, Insert, Update, Delete.	Provides a structured interface for managing and sharing an application's data with other applications securely. Uses URIs to identify data.
OkHTTP	Client, Request, Response.	Modern library for making HTTP requests. Request: Defines the URL, headers, and body. Response: Contains the server's reply (status code, body).
JSON/XML Parsing	Serialization/Deserialization.	Process of converting network data (JSON/XML string) into native Kotlin/Java objects, and vice versa.

6. Introduction to Flutter (Weeks 13, 14, 15)

Topic	Key Concepts	Notes for Viva
Flutter Core	Dart Language, Widgets.	Flutter uses the Dart language. UI is built entirely from a tree of Widgets.
Widget Types	StatelessWidget vs. StatefulWidget.	Stateless: Immutable, UI doesn't change after build (e.g., <code>Icon</code> , <code>Text</code>). Stateful: Manages mutable state via a separate <code>State</code> object (e.g., <code>Checkbox</code> , <code>Slider</code>).
Key Widgets	MaterialApp, Scaffold.	MaterialApp: The app's root, defining theme, title, etc. Scaffold: The standard screen template (<code>AppBar</code> , <code>Body</code> , <code>Drawer</code>).
Layouts	Column, Row, Container, Card.	Column/Row: Vertical/Horizontal arrangement (similar to Android Linear Layout). Container: Styling and positioning widget.
Routing	Navigator, Route.	Routers/Navigation manage the screen stack. <code>Navigator.push()</code> adds a new screen; <code>Navigator.pop()</code> removes the current screen.
State Management	Ephemeral State.	State that can be contained within a single widget (e.g., a simple counter). Managed locally within the <code>State</code> object using <code>setState()</code> .